24

BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

Phaser: Dinamik Thread Kaydı & Çok Fazlı İleri Düzey Bariyer

Phaser: Dynamic Thread Registration & Multi-Phase Barrier Architecture

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              PHASER: DİNAMİK VE ÇOK AŞAMALI BARİYER MİMARİSİ │
└──────────────────────────────────────────────────────────────┘

Phaser phaser = new Phaser(); // Başlangıçta 0 kayıtlı

phaser.register(); // Thread 1 dinamik olarak kendini kaydeder!
phaser.register(); // Thread 2 dinamik olarak kendini kaydeder!

FAZ 0 (Veri İndirme Aşaması):
Thread 1 ──► phaser.arriveAndAwaitAdvance() ──► Bekler...
Thread 2 ──► phaser.arriveAndAwaitAdvance() ──► HERKES GELDİ!
                                                    │
                                                    ▼ Faz 1'e Geçildi!

FAZ 1 (Veri Doğrulama Aşaması):
Thread 3 aramıza sonradan katılır! ──► phaser.register(); (Dinamik Kayıt!)
Thread 1 ayrılmak ister!           ──► phaser.arriveAndDeregister(); (Dinamik Çıkış!)

CountDownLatch ve CyclicBarrier'ın aksine: PARTİ SAYISI SABİT DEĞİLDİR!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir maraton düşünün. Ama bu maratonda yarışmacı sayısı baştan belli değildir! Koşu sırasında yeni koşucular yarışa dahil olabilir (`register()`), yorulan koşucular yarıştan çekilebilir (`arriveAndDeregister()`). Maraton 3 etaptan oluşur: 1. Etap (Faz 0), 2. Etap (Faz 1), 3. Etap (Faz 2). Her etabın kontrol noktasına gelen koşucu diğerlerini bekler; herkes geldiğinde hakem düdüğü çalar ve bir sonraki etap başlar! Phaser, katılımcı sayısı dinamik değişen çok etaplı bir yarış hakemidir!

🇬🇧 ENGLISH (ELI5)

Imagine a multi-stage endurance race where the participant roster is fluid. Mid-race, new runners can dynamically register (`register()`), and injured runners can withdraw (`arriveAndDeregister()`). The race spans discrete stages: Stage 0 (Mud Run), Stage 1 (Swim), Stage 2 (Sprint). At each checkpoint, runners wait until all registered participants arrive; the referee then advances the phase counter. Phaser is a dynamic, multi-phase CyclicBarrier supporting variable party counts.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Phaser vs CyclicBarrier / CountDownLatch: 1) Dinamik Katılımcı Sayısı (Dynamic Parties): CyclicBarrier'da katılımcı sayısı `new CyclicBarrier(N)` ile sabittir ve değiştirilemez. Phaser'da ise çalışma anında thread'ler `register()`, `bulkRegister(n)` veya `arriveAndDeregister()` ile serbestçe katılıp ayrılabilir. 2) Çok Fazlı Yapı (Phases): Phaser içsel olarak `getPhase()` ile artan bir faz sayacı tutar (0, 1, 2...). 3) Ağaç Yapılı Ölçekleme (Tiered Phasers): Yüzlerce thread'in tek bir senkronizöre saldırmasını önlemek için Phaser'lar hiyerarşik (Parent-Child) ağaçlar halinde kurulabilir; bu da devasa çok çekirdekli sunucularda çekişmeyi azaltır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Phaser Architecture & Scalability: 1) Dynamic Party Mutation: Unlike CyclicBarrier, party counts are mutable. Threads register on the fly (`register()`) and deregister upon early task completion (`arriveAndDeregister()`), preventing abandoned barriers. 2) Phase Counter Progression: Phaser tracks phase numbers monotonically via a 64-bit state field. `onAdvance(int phase, int registeredParties)` can be overridden to execute barrier logic or terminate the phaser. 3) Hierarchical Tree Phasers: To overcome memory bus contention across hundreds of cores, Phasers support hierarchical parent-child configurations, reducing synchronization contention via subtrees.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Phaser ile Dinamik Katılımlı Çok Fazlı İş Akışı
import java.util.concurrent.Phaser;

public class MultiPhaseTaskRunner {
    void main() {
        var phaser = new Phaser(1); // Main thread kendini kaydeder (Party: 1)

        for (int i = 1; i <= 2; i++) {
            phaser.register(); // Dinamik olarak yeni worker kaydet
            final int workerId = i;
            Thread.ofPlatform().name("Worker-" + workerId).start(() -> {
                IO.println("Worker " + workerId + " | Faz 0 (Veri İndirme) başladı.");
                phaser.arriveAndAwaitAdvance(); // Faz 0 tamamlandı, bekle!

                IO.println("Worker " + workerId + " | Faz 1 (Veri İşleme) başladı.");
                phaser.arriveAndAwaitAdvance(); // Faz 1 tamamlandı, bekle!

                phaser.arriveAndDeregister(); // İş bitti, kaydı sil ve ayrıl!
            });
        }

        IO.println("Main: Faz 0 bekleniyor...");
        phaser.arriveAndAwaitAdvance(); // Main thread Faz 0'ı onaylar

        IO.println("Main: Faz 1 bekleniyor...");
        phaser.arriveAndAwaitAdvance(); // Main thread Faz 1'i onaylar

        phaser.arriveAndDeregister(); // Main ayrılır
        IO.println("Tüm fazlar başarıyla tamamlandı!");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Sorusu: 'CyclicBarrier varken neden Phaser kullanmalıyız?' Yanıt: '1) Thread sayısı dinamik değişiyorsa, 2) Görevini erken bitiren thread'lerin havuzdan düşmesi gerekiyorsa, 3) Hiyerarşik ağaç yapısıyla binlerce thread'e ölçeklenmek gerekiyorsa Phaser tercih edilir.'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Interview Comparative Analysis: 'Why select Phaser over CyclicBarrier?' Senior Answer: '1) Dynamic thread registration/deregistration at runtime. 2) Programmable termination conditions via `onAdvance()`. 3) Tiered hierarchical tree structures to mitigate cache coherency contention under extreme thread counts.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Phaser, çalışma anında dinamik thread kaydına (register/deregister) izin verir.
- getPhase() ile aşama aşama ilerleyen çok adımlı iş akışlarını koordine eder.
- Hiyerarşik (Parent-Child) Phaser yapıları binlerce çekirdeğe ölçeklenebilir.

🎯 Key Takeaways for the Interview

- Phaser supports dynamic registration and deregistration of parties at runtime.
- Tracks sequential workflow phases via an incrementing phase counter.
- Hierarchical phasers eliminate bus contention across massive core counts.